Max Subset Sum No Adjacent

AlgoExpert · Medium · Dynamic Programming

The problem

- Max sum of positive ints with no two chosen elements adjacent.

Intuition

- At each index, either take it plus the best up to two back, or skip it.

Brute force vs optimal

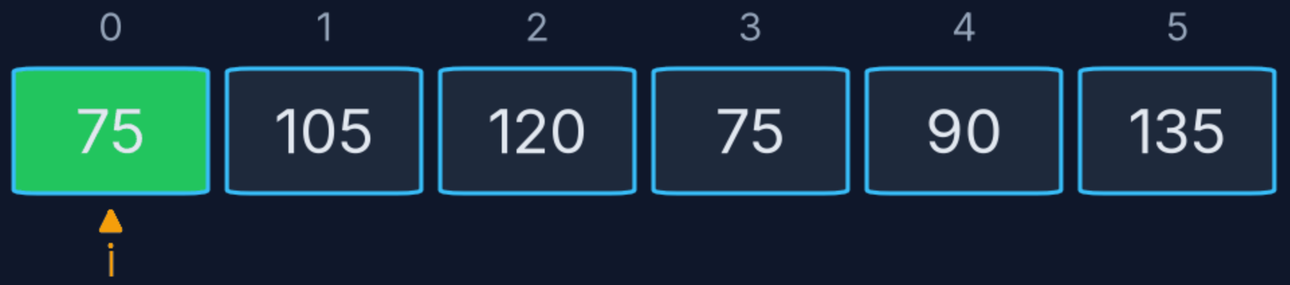
- Brute force: all subsets without neighbours → exponential
- DP array best[i] → O(n) time, O(n) space
- Only two previous values needed → O(1) space

Approach

- 1. Keep the best up to i minus one and i minus two
- 2. Choose the max of skip or take
- 3. Slide forward

Worked example

values



best sum up to i

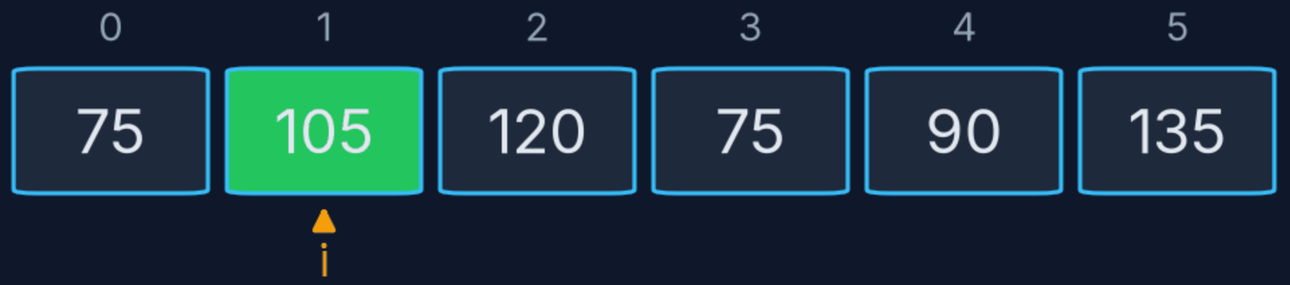



```
max(skip 0, take 0 + 75)
```

best[0] = 75 (take 75)

Worked example

values



best sum up to i

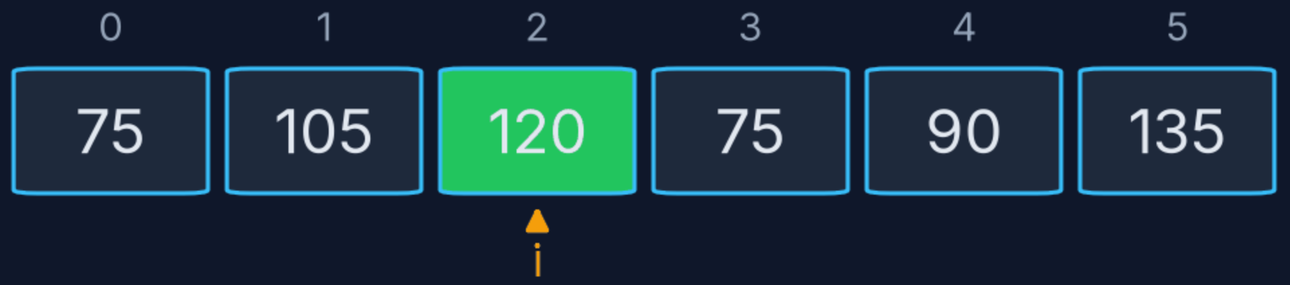



```
max(skip 75, take 0 + 105)
```

best[1] = 105 (take 105)

Worked example

values



best sum up to i

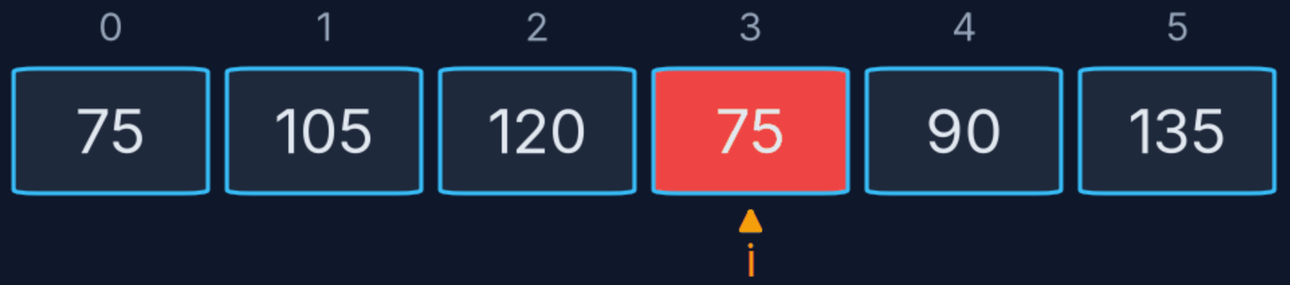



```
max(skip 105, take 75 + 120)
```

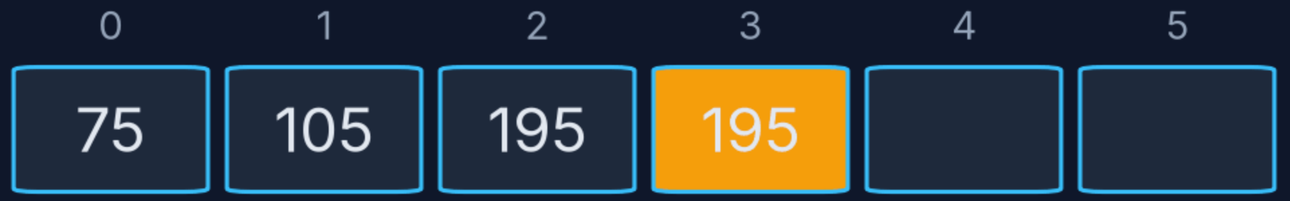
best[2] = 195 (take 120)

Worked example

values



best sum up to i

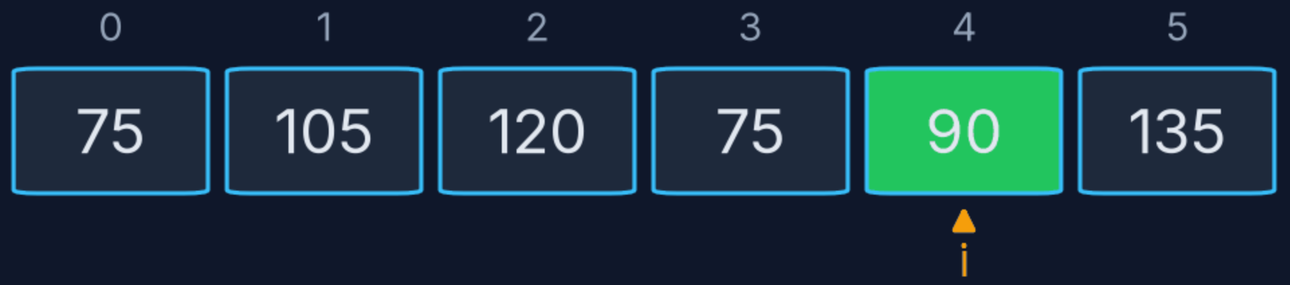



```
max(skip 195, take 105 + 75)
```

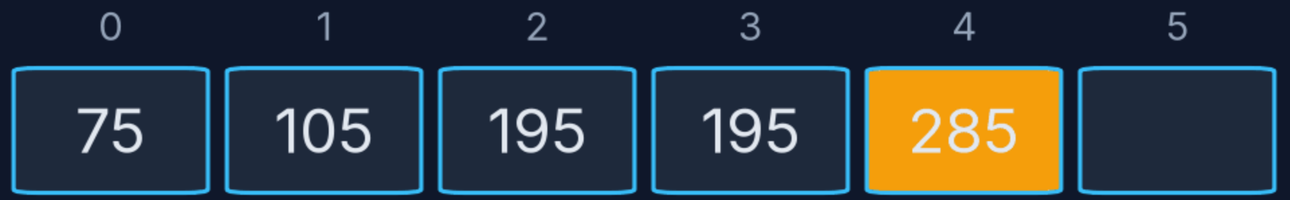
best[3] = 195 (skip 75)

Worked example

values



best sum up to i

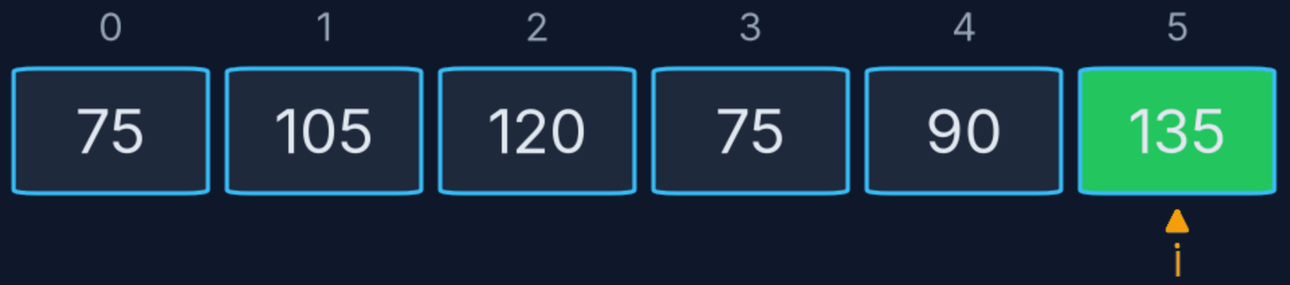



```
max(skip 195, take 195 + 90)
```

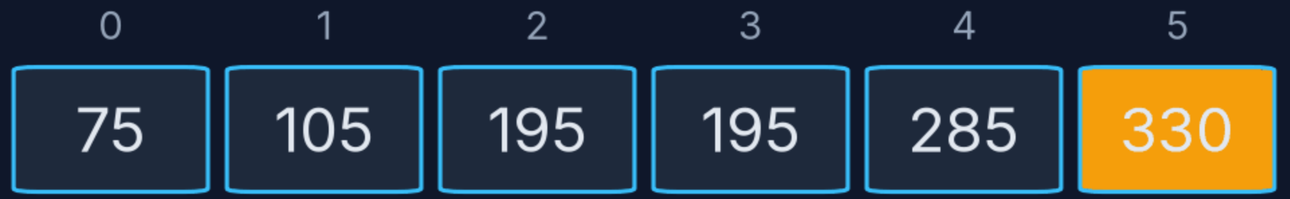
best[4] = 285 (take 90)

Worked example

values



best sum up to i




```
max(skip 285, take 195 + 135)
```

best[5] = 330 (take 135)

Java solution

```java
public class MaxSubsetSumNoAdjacent {
    public static int maxSubsetSumNoAdjacent(int[] a) {
        int prev = 0, cur = 0;
        for (int x : a) { int next = Math.max(cur, prev + x); prev = cur; cur = next; }
        return cur;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Empty array → 0
- Single element → itself
- Negative numbers: allow choosing nothing (0)

Interview tips & follow-ups

- This is House Robber
- Follow-up: circular array (run twice)

Recap

- At each index, either take it plus the best up to two back, or skip it.
- Complexity: O(n)